

UNIVERSIDAD DE MONTERREY
Escuela de Ingeniería y Tecnología
Ingeniería en Tecnologías Computacionales

AI Changelog

Autor(es)
600046 Fernando Olivares Del Valle

Profesor (a)
Prof. Raúl Morales Salcedo

Doy mi palabra que he realizado esta actividad con integridad académica

San Pedro Garza García, N.L. 27 de Agosto, 2026

AI Changelog

El prompt maestro del proyecto especifica los requisitos esenciales y completos de la aplicación, en donde se incluye las especificaciones del patrón y arquitectura a usar. Este documento usa el prompt maestro únicamente como contexto de referencia para el proyecto a desarrollar.

Prompt maestro:

None

1. Desarrolla una aplicación web monolítica en Node.js en /apps/web_monolito/ que gestione una librería en línea mediante acceso directo a PostgreSQL. La solución deberá renderizar HTML del lado del servidor, administrar usuarios registrados, implementar CRUD del modelo normalizado (en todas las tablas), manejar imágenes y conservar definiciones de conceptos asociadas a cada libro.
2. Restricción arquitectónica: no se desarrollarán APIs REST, GraphQL, SOAP ni otros servicios. No se utilizará JSON o XML como formato de intercambio de datos. El archivo package.json existe únicamente porque npm lo requiere para administrar el proyecto Node.js
3. Partiendo la base que todo libro tiene de ISBN, título, autor, año de publicación, género, precio, stock, formato, imágenes y conceptos definidos por libro, identifica dependencias funcionales y multivaluadas.
4. Un libro puede tener varios autores.
5. Un libro puede pertenecer a varios géneros.
6. Un libro puede definir muchos conceptos y un mismo concepto puede aparecer en distintos libros con definiciones diferentes.
7. Un libro puede tener varias imágenes.
8. Formato y categoría son catálogos independientes.
9. Debe existir como máximo un administrador
10. Utiliza la macro-arquitectura monolítica para el desarrollo del sistema
11. Utiliza el patrón de diseño MVC para la UI
12. Utiliza el enfoque de organización de código por módulos
13. Utiliza el esquema de base de datos para postgresql data/schema.sql
14. Crea un archivo README.md con las instrucciones de despliegue en Linux Centos 10 stream, asumiendo que tengo instalado el DBMS de Postgresql con el usuario:library_user password: 666 y base de datos library

Mejora 1: Validación real de contenido en la carga de imágenes

Al realizar las pruebas, se encontró que el sistema aceptaba un archivo de texto renombrado como .jpg como si fuera una imagen válida. La validación solo verifica la extensión del archivo y no el contenido.

Prompt utilizado:

None

Necesito que realices un cambio para poder detectar la carga de archivos en el campo de subida de imágenes. Al subir un archivo .txt renombrado a .png, este es aceptado. Debería de mostrar un mensaje de error.

Respuesta de la IA:

La IA identificó que los navegadores determinan el mimetype reportando según la extensión del archivo, y no por su contenido. Se propuso y aplicó una validación adicional que lee los primeros 12 bytes del archivo y verifica su firma binaria real contra las firmas de png, jpeg, webp. Si no corresponde, se elimina, se rechaza y se muestra un mensaje de error.

Archivos modificados

- src/controllers/bookController.js — se agregó la función esImagenValida() (lectura de magic bytes) y se modificó addImage() para validar el contenido real del archivo antes de persistir la referencia en la base de datos.

Riesgos introducidos

No hubo ningún riesgo introducido, puesto que se añadió una validación adicional. Sin embargo. Se mitigó por completo el riesgo.

Pruebas ejecutadas

Se creó un archivo .txt en donde se le modificó la extensión, asignando la de una imagen. Entre los resultados obtenidos está que se rechazó el archivo puesto que no era una imagen válida.

Mejora 2: Alertas tipo modal para retroalimentación de errores

Antes, cuando ocurría un error (por ejemplo intentar crear un segundo administrador, o subir una imagen inválida), la aplicación mandaba al usuario a una página aparte de error, perdiendo el contexto de dónde estaba parado.

Prompt utilizado:

None

Agrega alertas como si fueran modales para que el usuario tenga una mejor retroalimentación de los errores, en vez de mandarlo a una página de error completa.

Respuesta de la IA:

La IA modificó el manejador de errores central para que, en vez de renderizar una página de error nueva, redirija de regreso a la misma página donde estaba el usuario, mandando el mensaje de error codificado en la URL. Se agregó un modal (con su overlay y estilos) que se incluye en todas las vistas, y un script que revisa si la URL trae ese mensaje de error al cargar la página; si lo encuentra, muestra el modal encima del contenido, y al cerrarse limpia la URL para que no vuelva a aparecer si se recarga la página.

Archivos modificados

- `src/app.js` — se cambió el manejador de errores para redirigir con el mensaje en vez de renderizar `error.ejs`.
- `src/views/partials/foot.ejs` — se agregó el HTML del modal y el script que lo muestra/cierra.
- `public/css/app.css` — se agregaron los estilos del modal.

Riesgos introducidos

No se introdujo ningún riesgo.

Pruebas ejecutadas

Entre las pruebas ejecutadas, al querer crear un segundo administrador, aparece el modal y además se redirige a la página en donde se encontraba actualmente el administrador.